Architecture et Principes Fondamentaux des Systèmes d'Exploitation : Noyau, Appels Système et Gestion des Processus

Thème principal : Systèmes d'exploitation & Fonctionnement Bas Niveau

Synthèse exécutive : Ce cours aborde le rôle fondamental d'un système d'exploitation (OS) en tant qu'intermédiaire logiciel et gestionnaire de ressources entre le matériel (hardware) et les applications utilisateur. Nous étudions la frontière stricte entre l'Espace Utilisateur (User Space / Mode Non Privilégié) et l'Espace Noyau (Kernel Space / Mode Privilégié), le fonctionnement des interruptions et des Appels Système (Syscalls), ainsi que la gestion du cycle de vie des processus et leur ordonnancement par le noyau (Scheduling).

1. Le Rôle du Système d'Exploitation et la Séparation Mode Utilisateur / Mode Noyau

Un système d'exploitation est la couche logicielle fondamentale qui abstrait la complexité du matériel (processeur, RAM, disques, interfaces réseau) pour offrir un environnement d'exécution unifié et sécurisé aux logiciels. Pour garantir la stabilité globale de la machine et éviter qu'un programme défectueux ou malveillant ne détruise le système, le processeur physique prend en charge au moins deux modes de fonctionnement : le Mode Utilisateur et le Mode Noyau. En Mode Utilisateur, une application s'exécute avec un jeu d'instructions restreint et n'a accès qu'à son propre espace de mémoire virtuelle. Elle ne peut pas communiquer directement avec le matériel. En Mode Noyau, le système d'exploitation possède un accès total et illimité à l'ensemble du processeur, des registres matériels et de la mémoire RAM physique. Toute tentative d'accès non autorisé depuis le mode utilisateur déclenche immédiatement une faute matérielle.

- Système d'Exploitation (OS) : Couche d'abstraction logicielle et gestionnaire centralisé des ressources matérielles.
- Dualité de Mode d'Exécution : Mode Utilisateur (Ring 3 / non-privilégié) versus Mode Noyau (Ring 0 / privilégié).
- Isolation Mémoire : Protection matérielle empêchant une application d'altérer la mémoire d'autres programmes ou du noyau.

Cas concret enseignant : *Le professeur a utilisé la métaphore du restaurant : les applications en mode utilisateur sont comme des clients assis en salle. Ils n'ont pas le droit de s'introduire directement en cuisine pour manipuler les fourneaux ou piocher dans le réfrigérateur (le matériel). Ils doivent impérativement passer commande auprès du serveur (le noyau). Si un client tente de forcer la porte de la cuisine, le vigile du restaurant (la protection matérielle du CPU) l'expulse instantanément du bâtiment (Erreur de segmentation / Segmentation Fault).*

2. Les Appels Système (Syscalls) et la Commutation de Contexte

Lorsqu'une application a besoin d'interagir avec l'extérieur (ex: lire un fichier sur disque via read(), écrire du texte sur la console via write(), ou allouer de la mémoire via sbrk()), elle exécute une fonction de bibliothèque qui déclenche un Appel Système (Syscall). L'exécution d'une instruction processeur spéciale (comme `syscall` en x86_64) génère une interruption logicielle. Le CPU interrompt l'application, sauvegarde l'état courant des registres et passe en Mode Noyau (Ring 0). Le noyau traite la demande, vérifie scrupuleusement les droits d'accès du processus, communique avec le pilote matériel concerné, puis restitue le contrôle au programme en redescendant en Mode Utilisateur. Ce processus requiert une 'commutation de contexte' (Context Switch). Bien que rapide, le changement de mode et la sauvegarde des registres consomment du temps CPU, d'où l'importance de minimiser les appels système inutiles.

- Appel Système (Syscall) : Passerelle contrôlée permettant à un programme utilisateur de solliciter un service du noyau.

• Interruption Logicielle (TRAP / Instruction Syscall) : Mécanisme matériel basculant le processeur du mode utilisateur au mode noyau.

• Commutation de Contexte (Context Switch) : Sauvegarde de l'état d'un processus et chargement de l'état d'un autre, induisant un surcoût temporel (overhead).

Cas concret enseignant : *Le professeur a cité l'exemple d'un étudiant qui avait écrit une boucle écrivant 1 million de fois un seul caractère à l'écran avec l'appel système write() : l'exécution prenait plus de 15 secondes à cause d'un million de commutations de contexte ! En accumulant les caractères dans un tampon (buffer) en mémoire utilisateur et en n'effectuant qu'un seul appel système write() pour tout le paquet, le temps d'exécution est tombé à 0,002 seconde.*

3. Gestion des Processus, États et Ordonnancement (Scheduling)

Un processus représente un programme en cours d'exécution. Pour chaque processus, le noyau maintient une structure d'information appelée BCP (Bloc de Contrôle de Processus ou PCB) stockant son identifiant unique (PID), son état, la valeur de son pointeur d'instruction (PC), le pointeur de pile et la liste des ressources allouées. À tout instant, un processus se trouve dans l'un des trois états fondamentaux : Élu (il occupe le processeur et exécute ses instructions), Prêt (il possède toutes les ressources nécessaires mais attend que l'ordonnanceur lui attribue le CPU), ou Bloqué (il est en attente d'un événement externe, comme une lecture sur le disque ou une saisie utilisateur). L'Ordonnanceur (Scheduler) utilise des algorithmes comme le Tour de Rôle (Round-Robin) pour découper le temps CPU en petites tranches (quantums) et passer rapidement d'un processus prêt à un autre, donnant l'illusion d'une exécution simultanée (temps partagé / multitâche).

• Processus vs Programme : Un programme est une entité passive stockée sur disque ; un processus est une entité active chargée en mémoire vive.

• Bloc de Contrôle de Processus (BCP / PCB) : Structure de données du noyau contenant la carte d'identité complète d'un processus (PID, registres, état, fichiers ouverts).

• États d'un Processus : Élu (Running), Prêt (Ready), Bloqué (Blocked / Waiting).

• Ordonnanceur (Scheduler) : Composant du noyau attribuant le temps processeur aux processus prêts selon un algorithme défini (ex: Tour de Rôle / Round-Robin).

Cas concret enseignant : *Pour faire comprendre la transition vers l'état Bloqué, l'enseignant a comparé le processeur à un guichet de poste hyper rapide. Si le client en cours de traitement (Processus Élu) se rend compte qu'il a oublié de signer son formulaire (attente d'une Entrée/Sortie), il ne bloque pas le guichet : le guichetier le renvoie s'asseoir dans la salle d'attente des dossiers incomplets (État Bloqué) et fait immédiatement passer le client suivant rangé dans la file d'attente principale (État Prêt -> Élu).*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi les applications ne peuvent-elles pas accéder directement au disque dur ou au réseau pour gagner en vitesse ?
Réponse du professeur : Si n'importe quelle application pouvait écrire directement sur le disque sans le contrôle centralisé du noyau, deux programmes écrivant en même temps détruiraient la structure du système de fichiers. De plus, cela permettrait à une application malveillante de lire en direct les mots de passe et données personnelles des autres programmes en mémoire.

Question étudiant : Quelle est la différence fondamentale entre un Processus et un Thread ?
Réponse du professeur : Un processus possède son propre espace d'adressage mémoire complètement isolé des autres processus. Un thread (ou fil d'exécution) est une sous-unité à l'intérieur d'un processus : plusieurs threads d'un même processus partagent le même espace mémoire (variables globales, code, tas), ce qui permet une communication très rapide mais demande une synchronisation rigoureuse pour éviter que deux threads ne modifient la même donnée en même temps.

■ Exercice & Démonstration d'Application

Énoncé : Dans un système monoprocésseur utilisant l'ordonnancement au Tour de Rôle (Round-Robin) avec un quantum de temps Q = 4 ms et une durée de commutation de contexte (overhead) C = 1 ms, trois processus P1, P2 et P3 arrivent à t = 0 ms dans cet ordre. Leurs temps d'exécution CPU requis sont : P1 = 9 ms, P2 = 5 ms, P3 = 2 ms. Déterminez la chronologie d'exécution, le temps de fin de chaque processus et le temps total perdu en commutations.

Résolution pas à pas :
Étape 1 : Tracer le déroulement temporel pas à pas : - t = 0 à 4 ms : P1 s'exécute (Quantum de 4 ms consommé, reste 5 ms pour P1). - t = 4 à 5 ms : Commutation de contexte (1 ms). - t = 5 à 9 ms : P2 s'exécute (Quantum de 4 ms consommé, reste 1 ms pour P2). - t = 9 à 10 ms : Commutation de contexte (1 ms). - t = 10 à 12 ms : P3 s'exécute pendant 2 ms et SE TERMINE à t = 12 ms ! (P3 a terminé ses 2 ms requises). - t = 12 à 13 ms : Commutation de contexte (1 ms). - t = 13 à 17 ms : P1 s'exécute (Quantum de 4 ms consommé, reste 1 ms pour P1). - t = 17 à 18 ms : Commutation de contexte (1 ms). - t = 18 à 19 ms : P2 s'exécute pendant 1 ms et SE TERMINE à t = 19 ms ! (P2 a terminé ses 5 ms requises). - t = 19 à 20 ms : Commutation de contexte (1 ms). - t = 20 à 21 ms : P1 s'exécute pendant sa 1 ms restante et SE TERMINE à t = 21 ms ! Étape 2 : Calcul des temps d'exécution utiles et de surcoût : - Temps CPU utile requis = 9 + 5 + 2 = 16 ms. - Nombre de commutations effectuées = 5 commutations de 1 ms (à t=4, t=9, t=12, t=17, t=19) = 5 ms. - Temps total écoulé sur la machine = 16 ms + 5 ms = 21 ms.

Résultat : Dates de fin d'exécution (Turnaround) : P3 se termine à t = 12 ms, P2 se termine à t = 19 ms, P1 se termine à t = 21 ms. Le temps total d'exécution est de 21 ms, dont 5 ms perdues en commutations de contexte (soit environ 23,8% du temps machine dédié au surcoût de l'ordonnanceur).

■ Auto-Évaluation Flash d'Examen

Q : Quel composant matériel du processeur permet d'interrompre l'exécution du mode utilisateur pour passer en mode noyau lors d'un appel système ?

R : Le mécanisme d'interruption logicielle (ex: l'instruction 'syscall' ou 'TRAP' gérée par l'unité de contrôle du CPU).

Q : Dans quel état se trouve un processus qui attend la fin d'une lecture de fichier sur un disque SSD ?

R : Il se trouve dans l'état Bloqué (Blocked / Waiting) car il ne peut pas poursuivre son exécution tant que l'opération d'Entrée/Sortie n'est pas terminée.

Q : Qu'est-ce qu'une erreur de segmentation (Segmentation Fault) ?

R : C'est un signal généré par le système d'exploitation lorsqu'un programme tente d'accéder à une zone de mémoire vive pour laquelle il ne possède pas les droits de lecture ou d'écriture, ce qui provoque la fin immédiate du processus.